ENGINEERING BLUEPRINT · PROTOTYPE REPORT

# ClearFile Clean-Room Tax Filing Assistant

A clean-room re-implementation of the consumer tax-filing workflow: verified TY2025–TY2026 federal parameters, a Material Design 3 interview interface, open return files, and a bring-your-own-key AI ladder that steps down automatically from the highest-performance free model.

**ClearFile Project — Prototype 0.1**

Clean-room · MD3 · BYO-key AI ladder · Open formats

NOVEMBER 2026 · REV A · NOT FOR FILING

# Table of Contents

1 Executive Summary 2

2 Clean-Room Legal Strategy 3

3 System Architecture 4

4 Platform & Toolchain Matrix 6

5 Tax Engine Design 7

6 UI/UX Design System 9

7 Data & File Formats 11

8 AI Assistance Layer 12

9 Compliance & Certification Path 13

10 Roadmap & Milestones 14

Appendix A - Verified Tax Parameters & Sources 15

Appendix B - Obtaining Free AI Keys 16

# 1 Executive Summary

ClearFile is a clean-room re-implementation of the consumer tax-filing workflow: a guided interview that turns plain-language questions into a computed federal return. It is designed to replicate the *functionality* and functional layout conventions of modern tax-preparation software while scrupulously avoiding any copying of protected creative expression, trade dress, or proprietary file internals. This document is the engineering blueprint for that system, delivered alongside a running web prototype that already demonstrates the core experience end to end.

**$5,836**
Sample return refund, computed exactly by the prototype engine and independently verified before delivery

**5 models**
AI provider ladder: highest-performance free model first, automatic step-down on failure

**2 tax years**
Verified TY2025 and TY2026 federal parameter sets (IRS Rev. Proc. 2025-32 / OBBBA)

The prototype proves five things. First, a simplified but honest federal engine can compute real returns with verified IRS parameters, and every result it produces is traceable to a documented source. Second, a Material Design 3 interview interface can feel familiar to any user of commercial tax software without imitating any vendor's visual identity. Third, a bring-your-own-key AI layer can offer guided chat, document auto-fill, and pre-submit review at zero recurring cost to the project, stepping down a five-model ladder as free-tier limits are hit. Fourth, open file formats can carry a full return losslessly. Fifth, an honest inspector can interact with undocumented proprietary files without pretending to compatibility it does not have.

It is equally important to state what this is not. ClearFile today is demonstration software: it models one simplified federal pipeline (wages, interest, dividends, the standard-versus-itemized decision, child tax credits including a simplified refundable portion, and payments) and computes to the whole dollar. It is not an IRS Authorized e-file Provider, it does not yet cover the full form graph, preferential capital-gain rates, AMT, self-employment tax, or any state return, and it must not be used to file a real tax return. Section 9 defines the certification path and Section 10 the roadmap that would take ClearFile from this prototype to filing-capable software.

# 2 Clean-Room Legal Strategy

Clean-room implementation means independently reimplementing *functionality* while deliberately avoiding the original's protected expression. United States copyright law draws the line for us: 17 U.S.C. 102(b) excludes from copyright any "idea, procedure, process, system, method of operation, concept, principle, or discovery." The step-by-step interview flow, the order of questions (status, income, deductions, credits, payments, review), the presence of a progress indicator, and the placement of file menus are methods of operation shared by every commercial competitor. These are freely replicable. What remains protected is the creative skin layered on top: exact wording, color schemes, typography, iconography, illustration style, and overall trade dress. ClearFile replicates the former and originals the latter.

The functional-versus-expressive split drives every design decision. ClearFile's interface follows industry-generic conventions in an original Material Design 3 skin: a teal seed palette, tonal surfaces, rounded large shapes, and its own wordmark. No Intuit assets are used, no marketing language is echoed, and no screen copies any vendor's layout beyond the functional skeleton common to H&R; Block, TaxAct, FreeTaxUSA, and the IRS's own DirectFile pilot. This is precisely the approach every legitimate competitor has taken.

Two ingredients are legally cleaner than the rest. First, IRS DirectFile: works of the United States federal government are generally not subject to domestic copyright under 17 U.S.C. 105, so its plain-language interview patterns are free to adopt. ClearFile treats DirectFile as a design reference only - one decision per screen, plain questions, neutral governmental tone - with no assumption of a live backend. Second, the published TXF interchange format, which enables data portability with other desktop tax software without touching proprietary containers.

The proprietary TurboTax .tax format is the one area requiring a separately scoped workstream. Reverse engineering for interoperability enjoys limited protection under 17 U.S.C. 1201(f) and case law such as *Sega v. Accolade*, but the protection is fact-specific, shrink-wrap license terms may purport to forbid it, and Intuit's format remains undocumented publicly. ClearFile's current build therefore ships an honest inspector - it recognizes ZIP-based containers, lists stored entries, imports ClearFile's own exchange containers, and clearly reports anything it cannot read - and labels any export with the .tax extension as an open container, not Intuit compatibility. Section 10 defines the guarded reverse-engineering phase that would precede any real .tax import.

### Legal risk register

| Risk | Legal basis | ClearFile approach |
|---|---|---|
| Trade-dress claim from overall similar look | Unregistered trade dress, 15 U.S.C. 1125(a) | Original MD3 skin, own wordmark and palette; functional conventions only, shared across the whole industry |
| Copyright in creative expression | 17 U.S.C. 102(a) vs 102(b) | No copying of text, icons, artwork, or exact layouts; functional methods of operation only |
| Proprietary .tax format access | 17 U.S.C. 1201(f) interoperability exception; EULA terms | Isolated reverse-engineering team and documented spec, with counsel review before any importer ships |
| Method patents on tax-prep workflows | 35 U.S.C. (utility patents) | Freedom-to-operate search and design-around before production release |
| IRS program-name confusion | Federal trademark considerations | No use of "DirectFile" or IRS marks in branding; descriptive references only |
| E-file provider obligations | IRS e-file program, Pub. 3112 | No transmission until the certification path in Section 9 is completed |

Table 1. Risk register guiding the clean-room boundary.

# 3 System Architecture

ClearFile is architected as a platform-agnostic core wrapped in thin native shells. The core owns everything that must behave identically everywhere: the tax engine and its year-parameter configs, the file-format layer, the AI provider ladder, and the interview state model. The shells own nothing but presentation and platform integration - windows, packaging, file dialogs, and lifecycle. This separation is what keeps a seven-platform matrix tractable: a calculation bug is fixed once, in the core, and every platform inherits it.

The delivered web prototype is the reference implementation of the core. Its TypeScript modules - tax engine, store, AI ladder, and file layer - map one to one onto the production components and were verified end to end with automated browser testing. For production, the recommendation is to compile the engine once into a shared Rust library exposed to every shell through WASM and FFI bindings, so that no platform ever re-implements arithmetic. Per-platform calculation drift is the classic failure mode of multi-platform tax software, and a single compiled core eliminates it by construction.

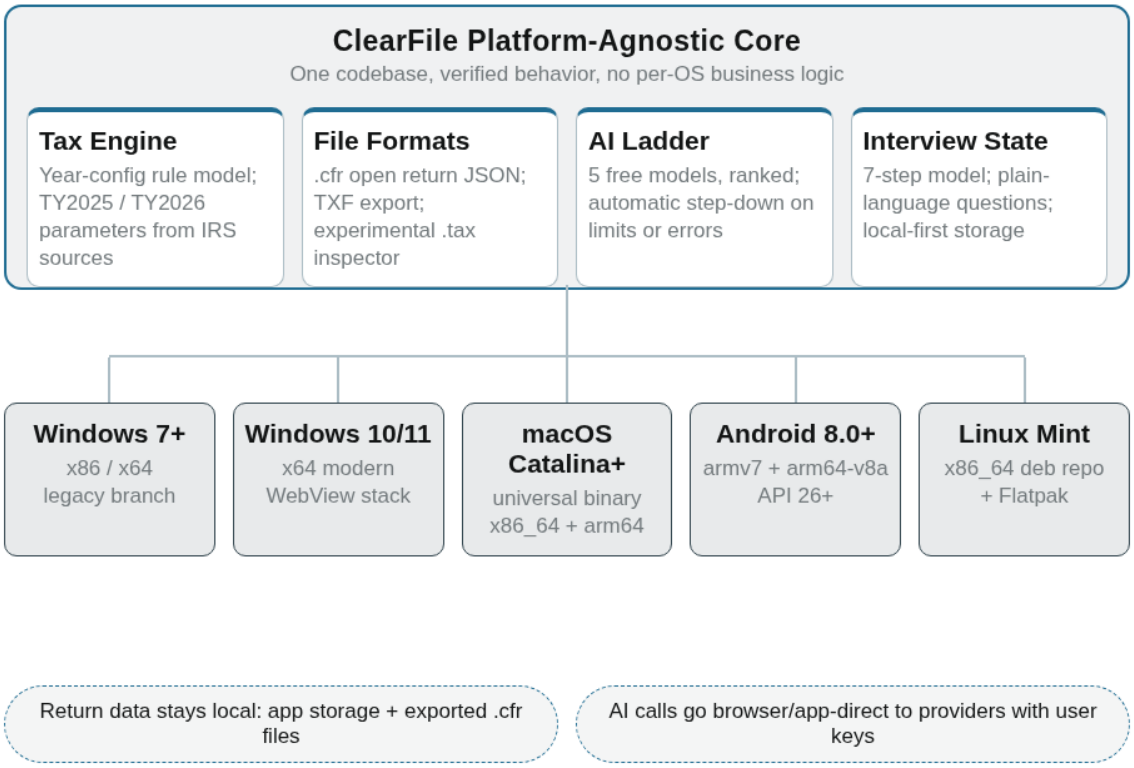



Figure 1. Shared-core architecture: one verified engine, five platform shells, local-first data.

## Core responsibilities and prototype status

| Module | Responsibility | Prototype status |
|---|---|---|
| Tax engine | Year-config rule evaluation, bracket walk, credits, balance; pure functions over immutable filing state | Implemented and verified (TY2025/TY2026, whole-dollar) |
| File layer | .cfr open return JSON, TXF export, experimental .tax inspector and container writer | Implemented; round-trip and inspector verified |
| AI ladder | Provider-agnostic task runner; five ranked free models with error classification and step-down | Implemented; live network calls and error paths verified |
| Interview state | Seven-step model, plain-language prompts, local-first persistence | Implemented (Zustand + localStorage) |
| Platform shells | Native windows, packaging, update channels | Blueprint only - Section 4 defines the matrix |

Table 2. Module responsibilities mapped to delivered prototype status.

Data flow is local-first by default. A return lives in browser or application storage and in files the user exports; nothing is uploaded to any server at any time. The only network traffic the prototype can generate is a direct browser-to-provider AI request, which occurs only after the user pastes their own API key, and even then the payload is a summarized return containing no names, SSNs, or addresses - ClearFile never collects them.

# 4 Platform & Toolchain Matrix

The requested platform matrix has a hard seam at Windows 7. Every modern first-party toolchain - Flutter's Windows runner, Tauri's WebView2 dependency, current Electron - requires Windows 10 or later. The honest answer is a two-tier Windows strategy: a frozen legacy branch for Windows 7 SP1 through 8.1 on both x86 and x64, and a modern branch for Windows 10 and 11. The legacy branch would ship from Electron 22, the final line with Windows 7 support, or a Qt 5.15 code path, and must carry an explicit security caveat: Windows 7 left extended support in January 2020, so users on that tier accept documented risk.

### Platform tiers

| Tier | Targets | Toolchain and packaging | Notes |
|---|---|---|---|
| Windows legacy | Win 7 SP1 - 8.1, x86 + x64 | Electron 22 branch or Qt 5.15; NSIS installer | Frozen toolchain; EOL security warning surfaced in-app |
| Windows modern | Win 10 / 11, x64 | Tauri 2 (WebView2) or current Electron; MSIX + NSIS | Primary Windows target; CI on both x64 machines |
| macOS | Catalina 10.15+, universal binary | Flutter 3.x (min 10.14) or Tauri; notarized .dmg | x86_64 + arm64 slices; hardened runtime entitlements |
| Android | 8.0 Oreo (API 26)+, armv7 + arm64-v8a | Flutter; AAB for Play, universal APK for sideload | Keep armv7 ABI alongside the required 64-bit |
| Linux | Mint 21+ (Cinnamon / MATE / Xfce), x86_64 | Flutter or Tauri; .deb repository + Flatpak on Flathub | Desktop-environment-agnostic; Flatpak covers non-Mint distributions |

Table 3. Platform matrix with honest toolchain constraints per tier.

Linux Mint's current variants differ only in desktop environment, which a GTK or Qt-based Flutter application is indifferent to; the deliverable is a single x86_64 build distributed through a .deb repository plus a Flatpak manifest for Flathub. Android Oreo sets the minimum SDK at 26, comfortably above Flutter's floor. macOS Catalina is supported by Flutter 3.x with a universal binary covering both Intel and Apple silicon. The repository already contains earlier Flutter scaffold iterations for Windows, macOS, Linux, and Android runners from prior development phases; they remain the starting point for the shell work in Phase 2.

# 5 Tax Engine Design

The engine is a pure function over an immutable filing state plus a year configuration. Year configs carry the standard deduction per filing status, the full bracket tables for all four statuses, and child-tax-credit parameters. Nothing about United States tax law is hard-coded into control flow, which is what makes annual updates - the IRS publishes its inflation adjustments every autumn via revenue procedure - a data change rather than a code change. All money is handled in whole dollars, matching return-level rounding.

Every parameter in the shipped configs was verified against primary or authoritative secondary sources in November 2026, and the sources are listed in Appendix A. Tax year 2026 follows Rev. Proc. 2025-32 as published on 9 October 2025; tax year 2025 reflects the One Big Beautiful Bill Act amounts. Two worked examples were recomputed by hand and matched the engine exactly: the sample married-filing-jointly return yields a $5,836 refund, and an imported single-filer return yields $3,220 of tax on $45,000 of wages with a $1,780 refund.

### Tax year 2026 brackets (taxable income)

| Rate | Single | Married joint | Head of household | Married separate |
|---|---|---|---|---|
| 10% | to $12,400 | to $24,800 | to $17,700 | to $12,400 |
| 12% | to $50,400 | to $100,800 | to $67,450 | to $50,400 |
| 22% | to $105,700 | to $211,400 | to $105,700 | to $105,700 |
| 24% | to $201,775 | to $403,550 | to $201,750 | to $201,775 |
| 32% | to $256,225 | to $512,450 | to $256,200 | to $256,225 |
| 35% | to $640,600 | to $768,700 | to $640,600 | to $384,350 |
| 37% | above | above | above | above |

Table 4. TY2026 bracket ceilings; lower bounds are the prior row ceiling. Source: IRS Rev. Proc. 2025-32.

### Standard deductions and family credits

| Parameter | TY2025 | TY2026 |
|---|---|---|
| Standard deduction - single / married separate | $15,750 | $16,100 |
| Standard deduction - married filing jointly | $31,500 | $32,200 |
| Standard deduction - head of household | $23,625 | $24,150 |
| Child tax credit, maximum per qualifying child | $2,200 | $2,200 |
| Refundable portion (ACTC) per child | $1,700 | $1,700 |
| CTC phase-out threshold (MFJ / other statuses) | $400,000 / $200,000 | $400,000 / $200,000 |
| Credit for other dependents | $500 | $500 |

Table 5. Standard deductions (OBBBA for 2025; Rev. Proc. 2025-32 for 2026) and family-credit parameters per IRS guidance.

The demo pipeline computes total income from W-2 wages, interest, ordinary dividends, and miscellaneous income; subtracts above-the-line adjustments to reach AGI; applies the larger of the standard deduction or itemized totals; walks the brackets; applies the child tax credit with its $50-per-$1,000 MAGI phase-out beyond $200,000 ($400,000 joint); computes a simplified refundable portion as 15 percent of earned income above $2,500, capped at $1,700 per child; adds the $500-per-dependent credit for other dependents; and nets withholding plus estimated payments into the final refund or balance due. Itemized amounts are deducted at face value in the demo - the SALT cap and the 7.5 percent medical floor are production rules, and the interface says so explicitly.

### Demo scope versus production engine

| Capability | Prototype | Production |
|---|---|---|
| Ordinary income, standard vs itemized, CTC/ACTC/ODC | Yes | Yes |
| Qualified dividends and LTCG preferential stacking | No | Phase 1 |
| SALT cap, medical floor, full Schedule A rules | No | Phase 1 |
| EITC, education credits, saver's credit | No | Phase 1 |
| AMT, NIIT, self-employment tax | No | Phase 1 |
| State returns (50 + DC) | No | Phase 2 |
| Full form graph and diagnostics | No | Phase 1-2 |
| IRS e-file transmission | No | Phase 5 (certification) |

Table 6. Honest scope accounting for the delivered engine versus the production roadmap.

# 6 UI/UX Design System

The interface is Material Design 3 with a teal seed palette, delivered through a complete token set: primary and secondary containers, tonal surface levels, outline and outline variant, plus success and error containers for refund-versus-owe semantics. Large 28-point corner radii on cards, fully rounded pill controls, and light/dark themes come straight from the MD3 specification. Accessibility is not decorative: all interactive targets meet or exceed 44 pixels, every control carries an accessible name, focus states are visible, and the whole flow was re-verified at a 390-pixel mobile viewport after the header was made responsive.

The layout skeleton is deliberately the industry-generic one - a left progress rail on desktop, a horizontal stepper on mobile, one decision per screen, plain-language questions in the DirectFile tradition, and a review screen where every number traces back to an entry. This is the functional convention layer that clean-room practice permits. Every visual property on top of that skeleton is ClearFile's own expression, summarized in Table 7.

### Conventions kept versus expression originated

| Element | Treatment | Rationale |
|---|---|---|
| Interview step order (status, income, deductions, dependents, payments, review) | Kept | Method of operation common to all competitors; user expectation |
| Progress rail / stepper presence | Kept (own skin) | Functional orientation device, independently skinned |
| One-decision screens with plain questions | Kept | DirectFile pattern; U.S. government work, public domain |
| Refund-versus-balance hero on results | Kept (own colors) | Functional result presentation; semantics via MD3 containers |
| Palette, typography, iconography, wordmark, microcopy | Original | Creative expression - never copied from any vendor |

Table 7. The clean-room boundary expressed as design decisions.

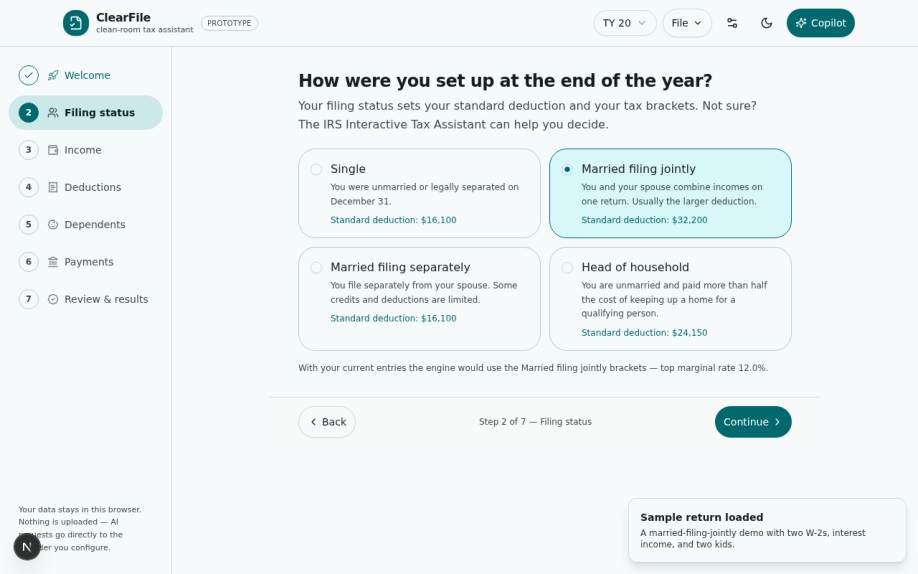



Figure 2. Filing-status step in the prototype: functional radio-card interview in the original MD3 teal skin.

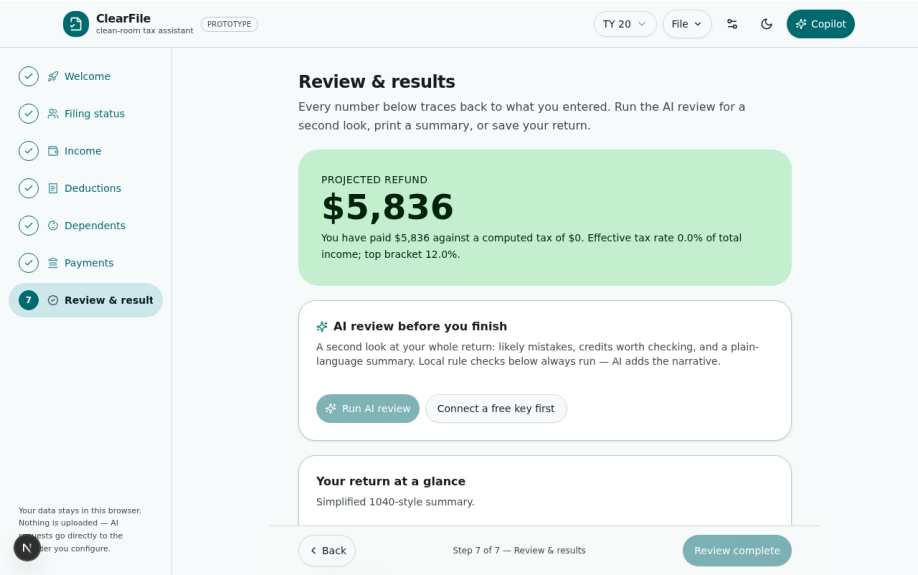



Figure 3. Review and results: verified refund stat, 1040-style line summary, bracket breakdown, and local rule checks.

Two further screens demonstrate the AI layer's integration points without visual disruption: the income step embeds the paste-to-auto-fill panel directly beneath the W-2 editor, and the results screen offers the AI review alongside the always-on local rule checks. The dark theme is a full MD3 dark scheme rather than an inversion, and the printable summary strips all application chrome so a user's printer output contains only the return.

# 7 Data & File Formats

ClearFile's native format is the ClearFile Return (.cfr): a versioned JSON document containing the format identifier, schema version, generating application, save timestamp, and the complete filing state. The schema is documented in the application source and this blueprint, and the loader validates every field, defaults missing values safely, and reports warnings rather than failing silently. Save and open were verified as a lossless round trip during end-to-end testing, including a return imported from a .cfr file that then computed to the exact expected figures.

For interchange with other desktop tax software, the prototype exports TXF, the published Tax Exchange Format. The current exporter writes the two record codes within the demo scope - code 425 for combined W-2 wages and code 426 for combined federal withholding - and the interface states plainly that imported results should be verified in the receiving application. Production would extend coverage across the schedule lines the engine supports.

The proprietary .tax container is handled with deliberate honesty. Intuit's format is undocumented; claiming compatibility without documentation would be dishonest. The shipped analyzer detects ZIP-family containers, enumerates stored entries with previews, recognizes and fully imports ClearFile exchange containers, reports deflate-compressed entries as present but not extracted, and returns a clear "unrecognized container" verdict with the blueprint's explanation for anything else. The exporter writes an open ZIP container under the .tax extension carrying clearfile.json plus a README that states it is not the Intuit format - a convenience for users who want one file per year, never a compatibility claim. All three analyzer paths were verified during testing.

### Format summary

| Format | Role | Status |
|---|---|---|
| ClearFile Return (.cfr) | Native save/open; documented open JSON schema | Delivered and verified |
| TXF | Interchange export to other desktop tax software | Delivered (wages + withholding codes); verify after import |
| .tax inspector | Analyze existing proprietary files (ZIP detection, entry listing, ClearFile container import) | Delivered and verified; honest non-claims for unknown containers |
| .tax exchange container | User convenience export using the extension | Delivered, labeled experimental; not Intuit format |
| Import plugin API | Third-party format adapters (production) | Specified; Phase 2 with the RE workstream |

Table 8. File-format strategy: open where possible, honest where not.

# 8 AI Assistance Layer

ClearFile's AI assistance is bring-your-own-key by design: the user obtains a free API key from Google AI Studio (the Gemini family) or Mistral's La Plateforme console, pastes it into Settings, and the application stores it only in local browser storage. Requests travel directly from the user's browser to the provider endpoint - there is no ClearFile server in the middle, and the prototype collects no names, SSNs, or addresses, so the context sent to models is a summarized return only. Step-by-step key instructions and direct links appear both in Settings and in Appendix B.

The provider ladder implements the product's core requirement: always try the highest-performance free model first, then step down when a model is inaccessible or its limits have been reached. Every call classifies its outcome - rate limit, authentication or quota rejection, model unavailable, network or CORS failure, or timeout - and the ladder moves to the next model on any failure, remembering the last model that answered. The classification was verified live during testing: an invalid Google key produced the correct "key rejected" status on all three Gemini models while Mistral models reported "no key" - exactly the designed behavior.

### Model ladder (tried in order)

| Order | Model | Provider | Role |
|---|---|---|---|
| 1 | Gemini 2.5 Pro | Google AI Studio | Highest performance free model; tightest limits, first to fall back |
| 2 | Gemini 2.5 Flash | Google AI Studio | Balanced reasoning and speed; generous free tier |
| 3 | Gemini 2.5 Flash-Lite | Google AI Studio | Fastest, most permissive Google fallback |
| 4 | Mistral Large | Mistral La Plateforme | Flagship fallback once Google limits are exhausted |
| 5 | Mistral Small | Mistral La Plateforme | Final lightweight fallback |

Table 9. The five-model performance ladder with automatic step-down.

### Error classification and ladder action

| Signal | Classification | Ladder action |
|---|---|---|
| HTTP 429 | Rate limit reached | Step down immediately; UI notes limits were hit |
| HTTP 400 / 401 / 403 | Key rejected or quota exceeded | Step down; user directed to check the key |
| HTTP 404 | Model unavailable for this key | Step down |
| Fetch TypeError | Network or CORS failure | Step down; Mistral browser-direct calls typically land here |
| AbortError | Timeout | Step down after 45 s |
| All models failed | Ladder exhausted | Per-model failure table shown with actionable hints |

Table 10. Every failure mode maps to a specific, tested behavior.

Three features ride on the ladder. The Copilot is a context-aware chat that answers questions about the return in plain language, with prompts that forbid inventing numbers and confine it to the summarized context. Auto-fill accepts pasted W-2 or 1099 text and returns structured forms for explicit user review - nothing is written to the return until the user selects and applies the parsed rows, which is the layer's most important guardrail. The pre-submit review pairs always-on local rule checks (no withholding recorded, itemized below standard, suspiciously large refund or balance) with an AI narrative of findings and a plain-language summary, always labeled as AI output that must be verified.

One limitation is documented rather than hidden: Mistral's API does not send CORS headers, so browser-direct Mistral calls fail as unreachable in the web prototype. The ladder treats this exactly like any inaccessible model, and the Settings screen says so. The production build adds an optional local proxy - trivial in the desktop shells, where the application itself can relay requests - which restores Mistral and also enables per-device key encryption. This is scheduled with the platform shells in Phase 2.

# 9 Compliance & Certification Path

ClearFile today is demonstration software and says so on the welcome screen, on the results screen, and in the footer of every page. It performs no transmission of any return to any authority, collects no taxpayer identifiers, and runs no telemetry. That posture is deliberate: the fastest way to poison a clean-room project is to imply filing capability that does not exist.

Becoming filing-capable is a defined path rather than an aspiration. The IRS e-file program requires an approved e-file application (Publication 3112 walks the process), completion of software-developer acceptance testing against IRS scenarios before live transmission, and continuing obligations including suitability checks and annual refresher testing. In parallel, any service that handles return data adopts a Safeguard posture aligned to IRS Publication 4557 and related standards - for ClearFile the local-first architecture is the head start, since stored returns live under the user's own device encryption and the only network egress is user-keyed AI traffic.

### Compliance milestones

| Requirement | Current status | Milestone |
|---|---|---|
| Demo disclaimers and no-filing posture | Delivered in prototype | Maintained until certification |
| Engine accuracy harness (golden returns) | Two hand-verified examples | Phase 1: automated corpus vs commercial references |
| Annual parameter updates | TY2025/TY2026 verified from IRS sources | Each autumn's revenue procedure tracked as a data PR |
| IRS e-file application and ATS testing | Not started | Phase 5 gate |
| Safeguard posture (Pub. 4557 alignment) | Local-first by design | Phase 3 with proxy and key encryption |
| State e-file participation (where required) | Not started | Post-federal certification |

Table 11. The certification path from prototype to filing-capable software.

# 10 Roadmap & Milestones

The roadmap sequences the remaining work so that risk retires early. Phase 1 completes the federal engine and, critically, builds the golden-return test harness that every later phase depends on. Phase 2 stands up the platform shells from the existing Flutter scaffolds and the compiled core. Phase 3 hardens the AI layer with the local proxy and key encryption. Phase 4 is the guarded reverse-engineering workstream for .tax import, which proceeds only after the legal review in Section 2 is satisfied. Phase 5 walks the IRS certification path. Estimates assume a small senior team and are stated as ranges because tax-software schedules reliably discover surprises.

### Phases

| Phase | Scope | Exit criteria | Estimate |
|---|---|---|---|
| 0 - Prototype (delivered) | Web reference implementation of the core | This document and the verified application | Complete |
| 1 - Engine completion | Preferential rates, SALT/medical rules, EITC, education credits, AMT/NIIT, SE tax; golden-return harness | Corpus agreement with commercial references within rounding | 2-4 engineer-months |
| 2 - Platform shells | Rust core + WASM/FFI; five shell targets; CI matrix | Green builds and released installers on all tiers | 3-6 engineer-months |
| 3 - AI hardening | Local proxy for CORS-free Mistral; key encryption; review escalation paths | Penetration-tested key handling | 1-2 engineer-months |
| 4 - .tax interop (gated) | Isolated RE team, documented format spec, importer | Counsel sign-off; round-trip test corpus | 2-3 engineer-months + legal |
| 5 - Certification | IRS e-file application, ATS scenarios, safeguards | Accepted as e-file software developer | 3-6 months elapsed, largely process-driven |

Table 12. Phased plan with exit criteria; Phase 4 is legally gated before any RE work.

# Appendix A - Verified Tax Parameters & Sources

All engine parameters were checked against the following sources in November 2026. Head-of-household TY2025 brackets were taken directly from the IRS page; TY2026 figures follow Rev. Proc. 2025-32 as published 9 October 2025 and cross-checked between the Tax Foundation and Fidelity's IRS-sourced tables. The TY2026 refundable child-tax-credit amount is applied as $1,700 per Jackson Hewitt's 2026 guidance; because every parameter lives in the year config, an updated indexed figure is a one-line data change. Tax year 2025 standard deductions reflect the One Big Beautiful Bill Act as reported by NTU and CPA Practice Advisor.

Primary and authoritative sources used:

IRS. "Federal income tax rates and brackets." irs.gov/filing/federal-income-tax-rates-and-brackets - TY2025 brackets including head of household.

IRS. "Child Tax Credit." irs.gov/credits-deductions/individuals/child-tax-credit - $2,200 maximum, $1,700 refundable, phase-out thresholds.

IRS. Rev. Proc. 2025-32 (TY2026 inflation adjustments), published 2025-10-09; as reported by CPA Practice Advisor, cpapracticeadvisor.com.

Tax Foundation. "2026 Tax Brackets." taxfoundation.org/data/all/federal/2026-tax-brackets.

Fidelity. "2025 and 2026 tax brackets and federal income tax rates." fidelity.com/learning-center/personal-finance/tax-brackets.

NTU Foundation. "What Are Federal Income Tax Rates for 2025 and 2026?" ntu.org/foundation/tax-page.

Jackson Hewitt. "Child Tax Credit 2026." jacksonhewitt.com/tax-help/tax-tips-topics/family/child-tax-credit-2026.

Statutes referenced: 17 U.S.C. 102, 105, 1201(f); 15 U.S.C. 1125(a); IRS Pub. 3112, Pub. 4557.

# Appendix B - Obtaining Free AI Keys

ClearFile ships with no bundled keys and no accounts. Users who want AI assistance obtain a free key from either provider below; the same instructions appear in the application's Settings dialog with clickable links.

**Google AI Studio (Gemini family).** Open aistudio.google.com/apikey and sign in with a Google account; click "Create API key" and pick or create a project; copy the generated key (it begins with "AIza"); paste it into ClearFile Settings. The free tier carries per-model rate limits, which the ladder treats as a normal step-down condition.

**Mistral (La Plateforme).** Open console.mistral.ai/api-keys and create an account; choose the free Experiment plan when prompted; click "Create new key," name it, and copy it into Settings. Browser-direct calls to Mistral are blocked by CORS in the web prototype - the ladder reports this honestly and the desktop build's local proxy removes the limitation.

Privacy notes shown in the application bear repeating: keys are stored only in the local browser, are transmitted only to the provider whose key they are, and clearing browser data removes them. AI requests see a summarized return without names, SSNs, or addresses, because ClearFile never collects those fields in the first place.

---

This blueprint describes prototype software. ClearFile is not IRS-approved tax preparation software and must not be used to file a real return. All tax parameters are stated as verified in November 2026 and are subject to annual IRS adjustment.